

Amazon SQS Hands-On Lab

To Begin with the Lab

Summary of the Lab

In this lab, an **Amazon SQS Standard Queue** was created and used with two **Amazon EC2** instances to demonstrate a complete pull-based messaging workflow. One EC2 instance acted as the **producer** and sent messages into the queue through a web UI, while the other EC2 instance acted as the **consumer** and received and deleted messages from the queue through a separate web UI. Both EC2 instances were given access to SQS using a shared **IAM role** attached through the instance profile, which followed AWS best practices and avoided hardcoded access keys. The lab also confirmed that the producer only pushes messages into SQS, while the consumer pulls messages from SQS, since SQS never pushes messages itself. Finally, the end-to-end flow was validated by sending a message, confirming the queue count increased, receiving the message, and then verifying that it was removed from the queue after polling.

Understanding Amazon SQS in a Producer-Consumer Workflow

Amazon SQS is a managed message queue that lets one system send messages and another system process them asynchronously. It exists so the producer and consumer do not need to talk to each other directly, which makes the system more flexible and reliable. In this lab, the producer sends a message into the queue, and the consumer later pulls that message and deletes it after processing. This pattern is useful when you want loose coupling between applications and a simple way to buffer requests.

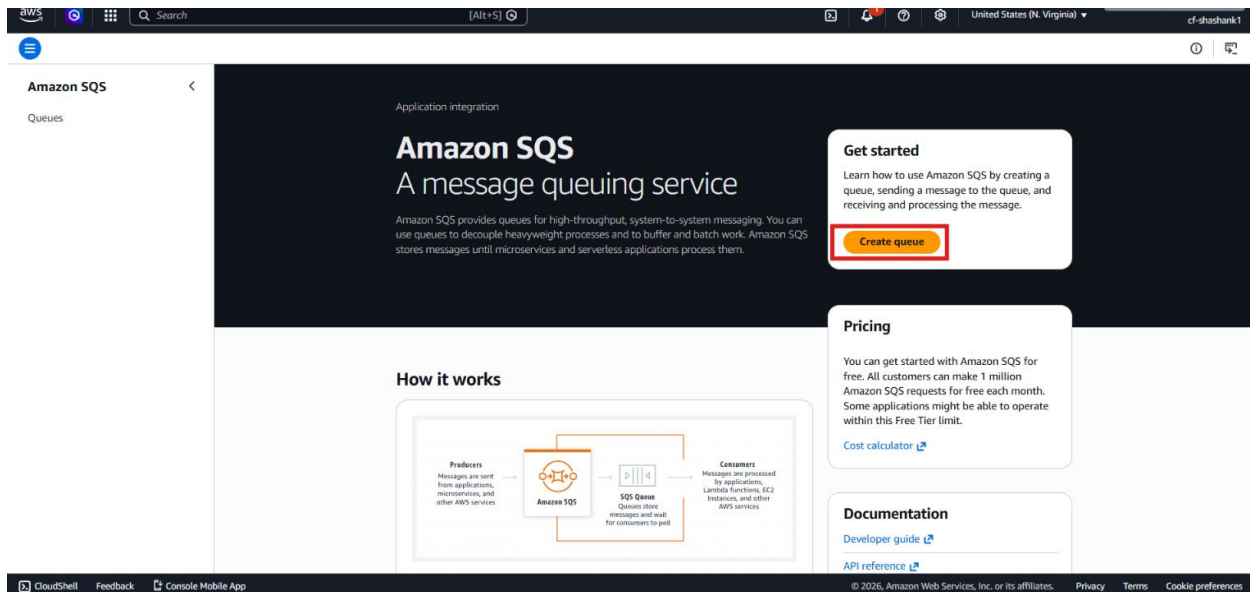
Example:

A web app receives an order message like OrderID-5001. The producer EC2 instance sends that order into the SQS queue, and the consumer EC2 instance later polls the queue and removes the message after reading it. That way the order is stored safely even if the consumer is busy for a while.

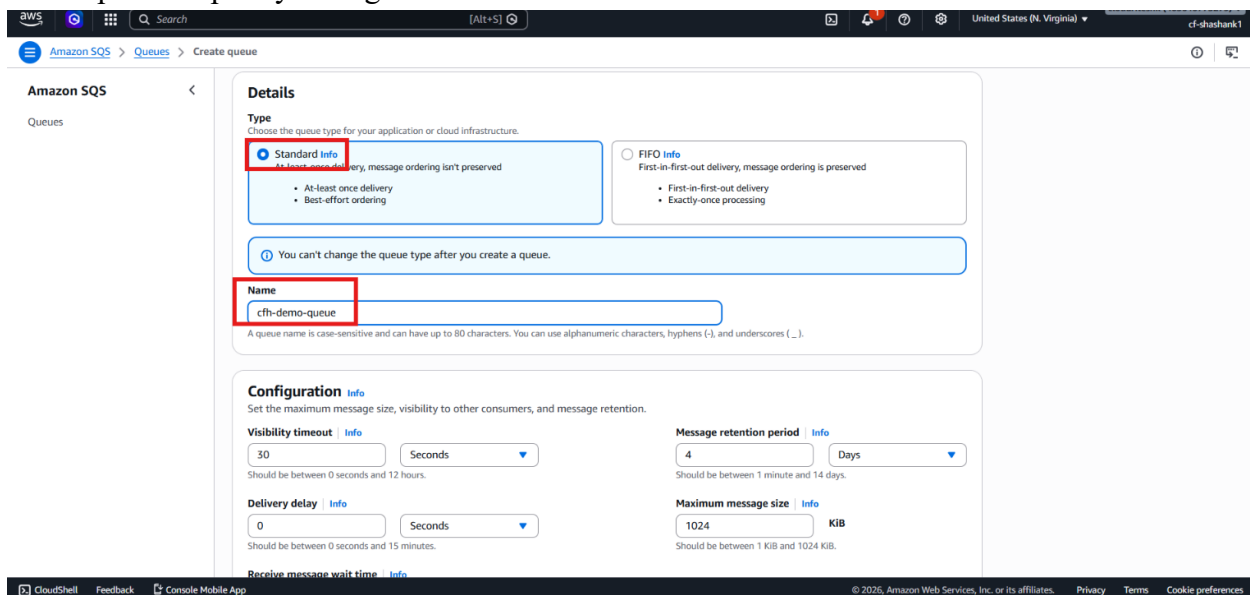
Lab Steps

Step 1 – Create the SQS Standard Queue

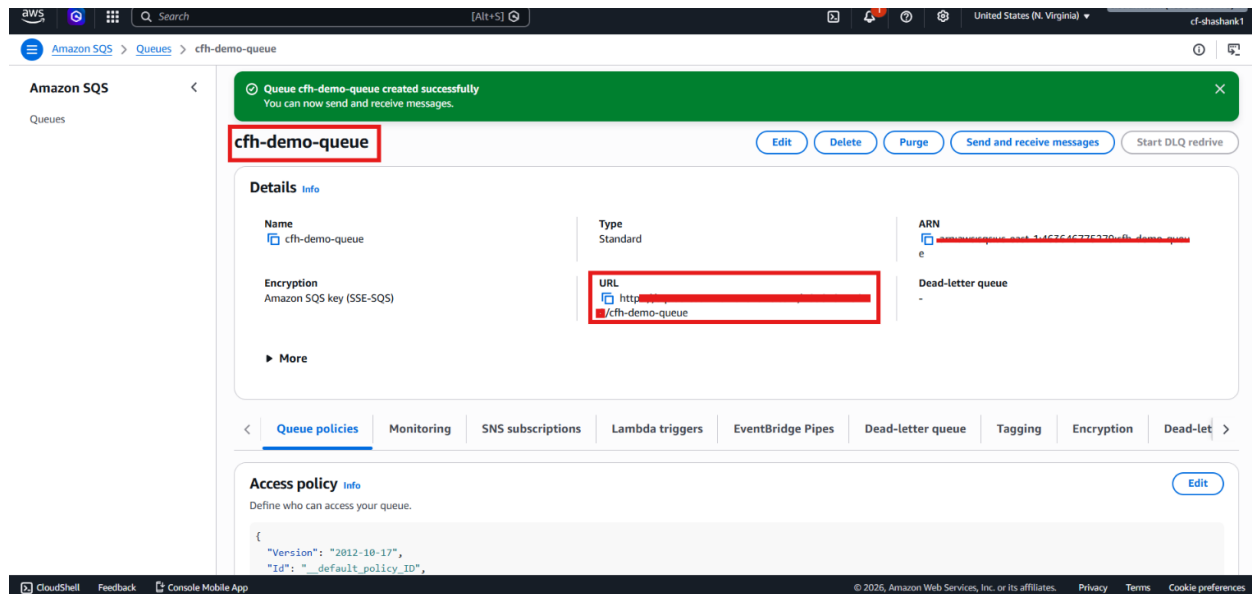
- Sign in to the AWS Management Console.
- Open **Amazon SQS**.
- Click **Create queue**.



- Select **Standard** queue type.
- Enter the queue name: cfh-demo-queue
- Keep the default settings.
- Leave encryption at the default setting.
- Skip access policy configuration for now.

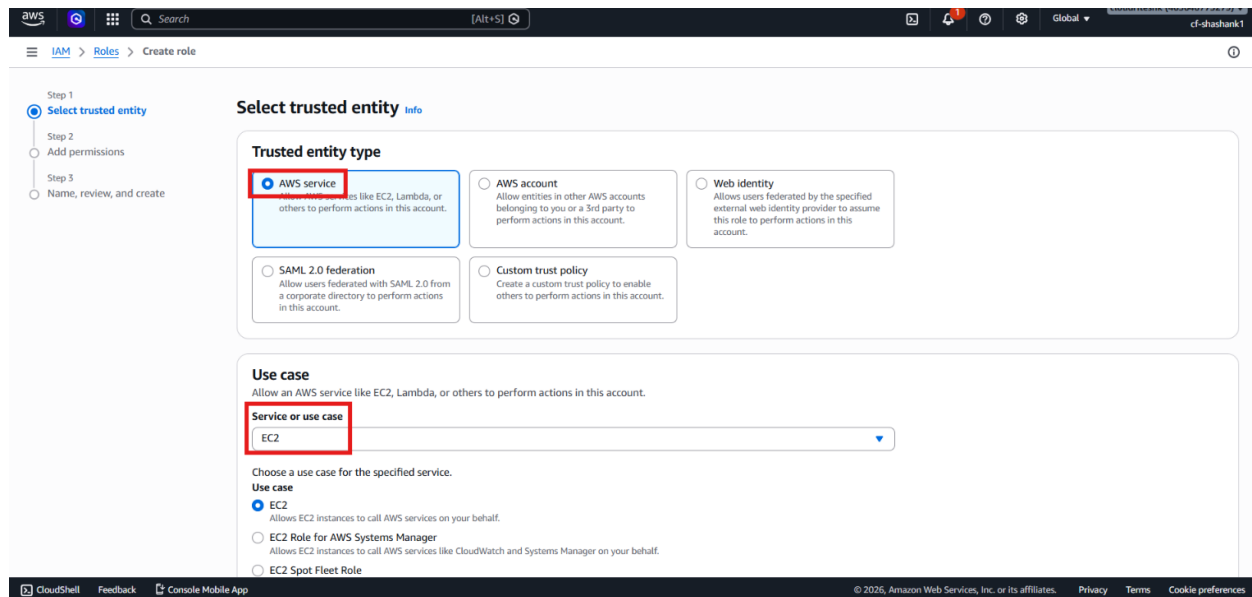


- Click **Create queue**.
- Copy the **Queue URL** for later use in the producer and consumer web UIs.

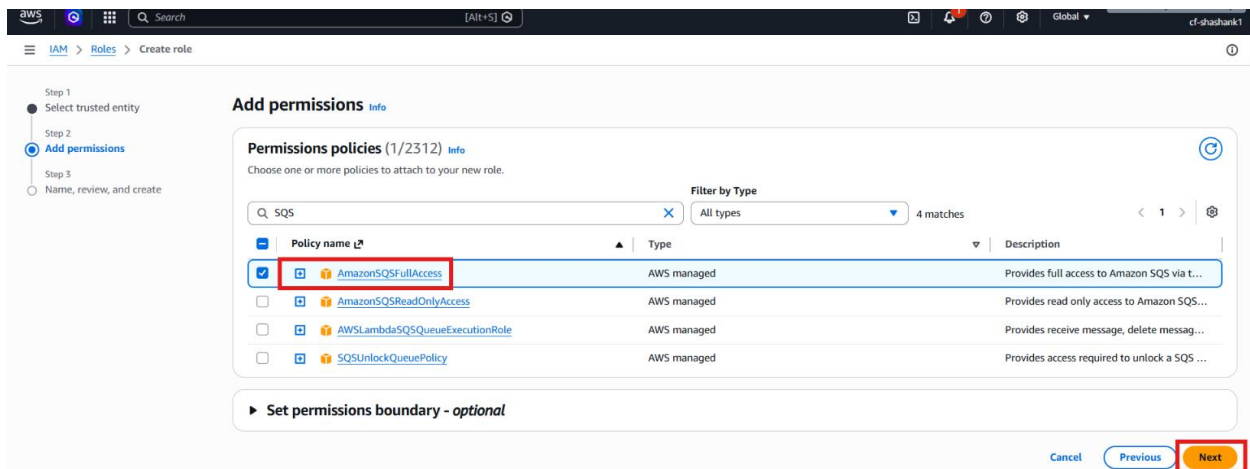


Step 2 – Create the IAM Role for EC2

- Open **IAM**.
- Click **Roles**.
- Click **Create role**.
- Choose **AWS service** as the trusted entity.
- Select **EC2** as the use case.



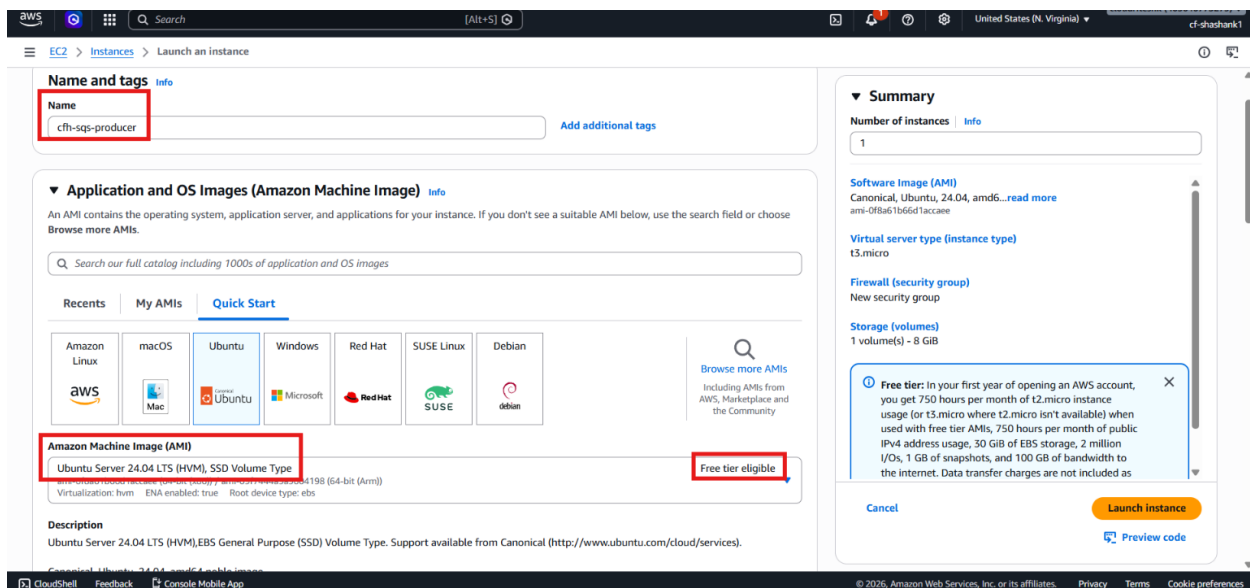
- Attach **AmazonSQSFullAccess**.
- Enter the role name: **CloudFreeks-SQS-EC2-Role**
- Create the role.



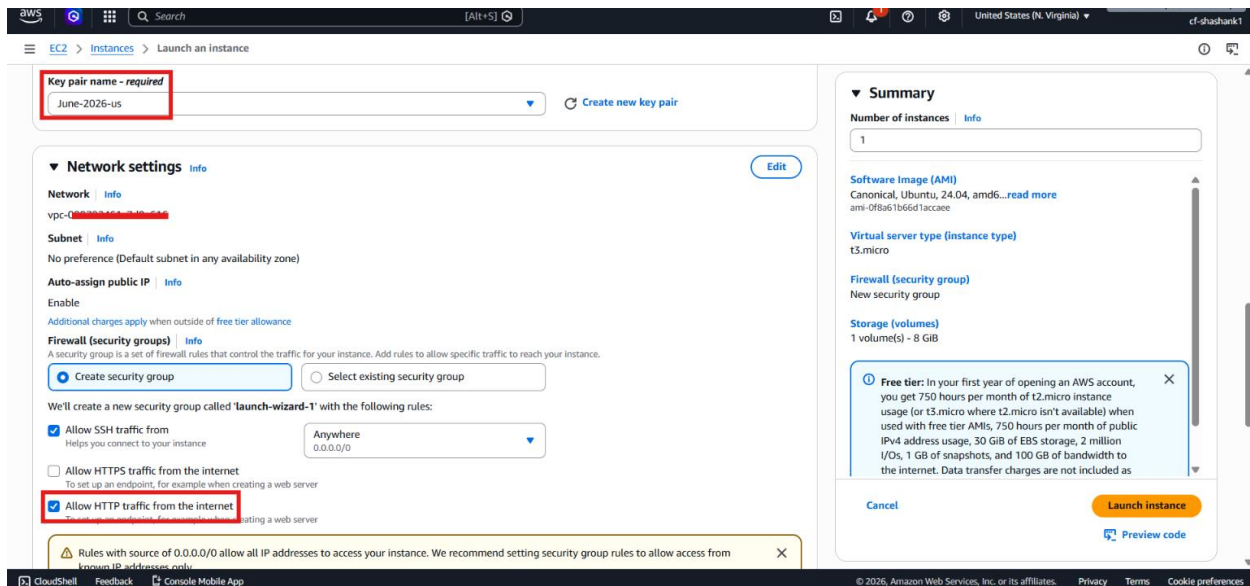
- Confirm that this role will be attached to both EC2 instances through the instance profile.

Step 3 – Launch the Producer EC2 Instance

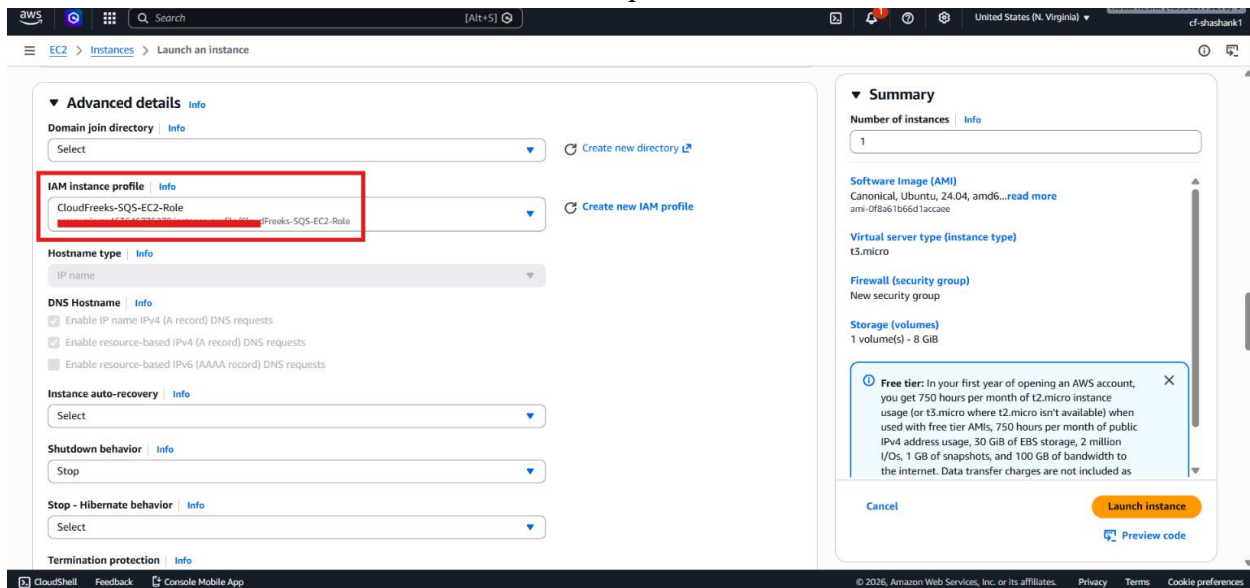
- Open Amazon EC2.
- Click **Launch instance**.
- Enter the instance name: cfh-sqs-producer
- Select **Ubuntu 24.04 LTS**.
- Choose **t2.micro** or **t3.micro**.



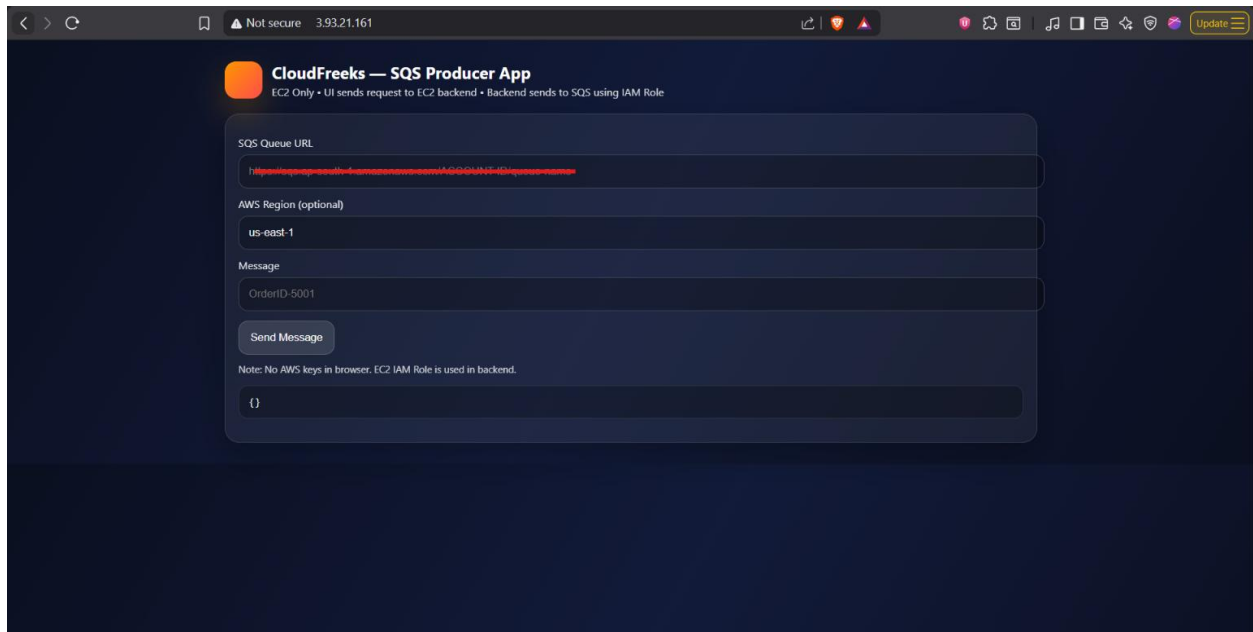
- Select or create a key pair.
- Use the default VPC.
- Allow inbound **SSH 22** and **HTTP 80**.



- In advanced details, select the IAM instance profile **CloudFolks-SQS-EC2-Role**.

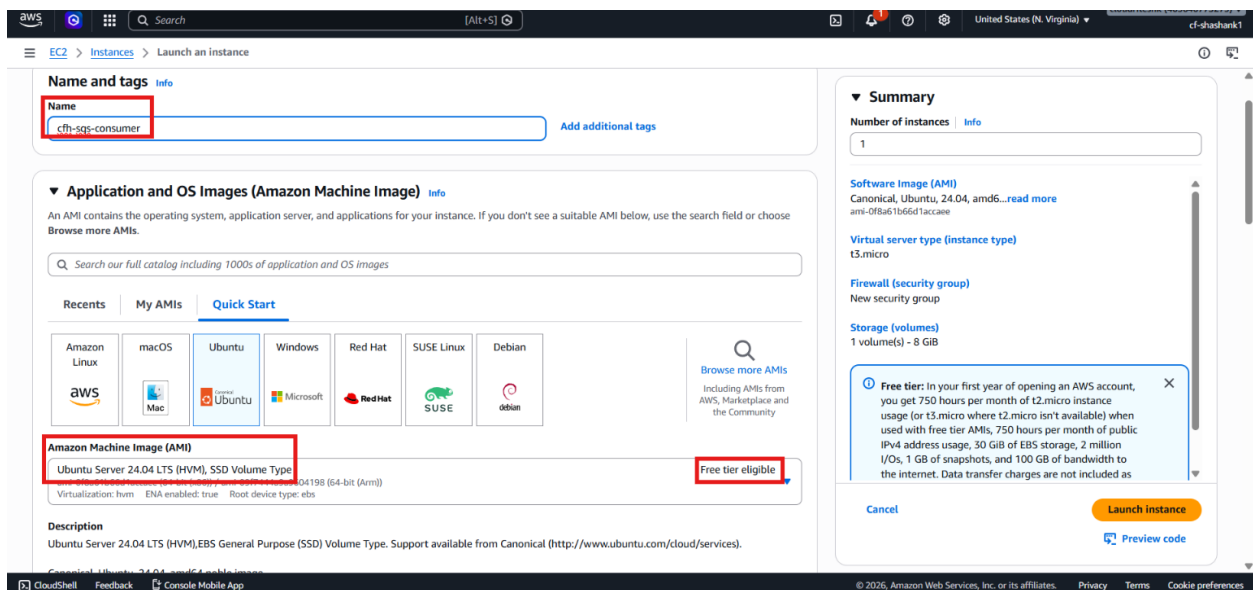


- Add the producer user data script.
- Launch the instance.
- Wait until the producer web UI is ready.

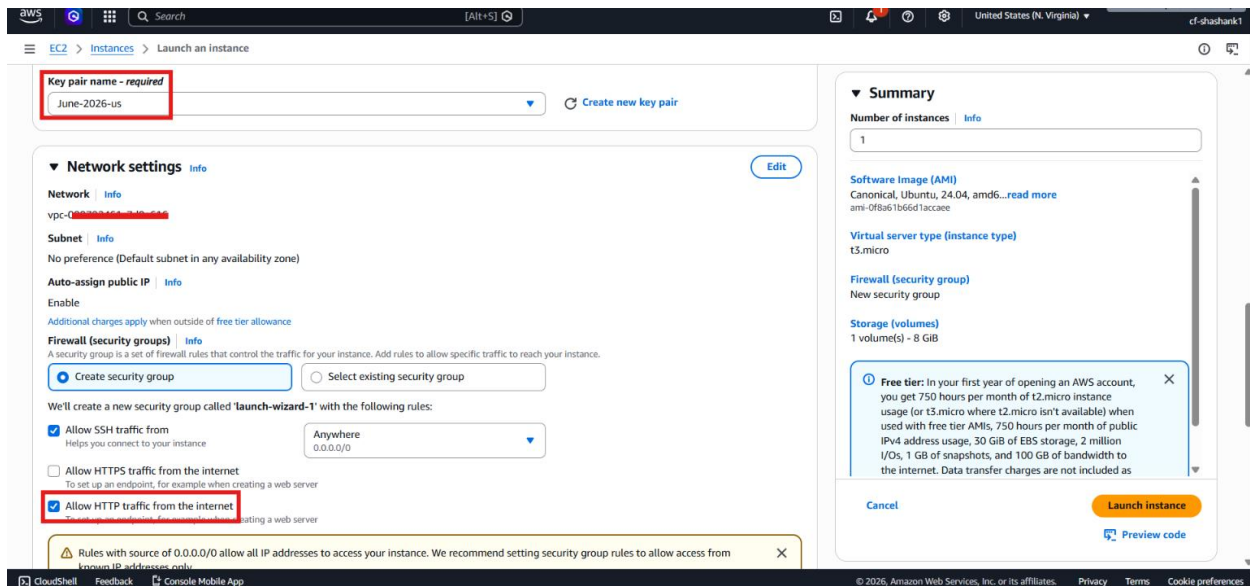


Step 4 – Launch the Consumer EC2 Instance

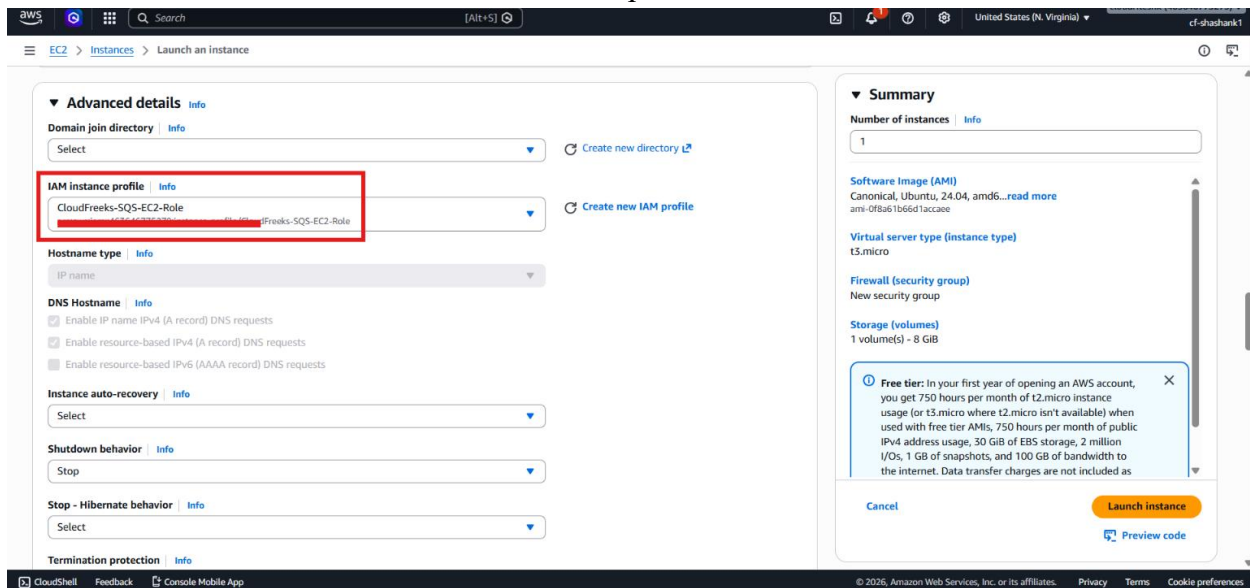
- Open **Amazon EC2**.
- Click **Launch instance**.
- Enter the instance name: **cfh-sqs-consumer**
- Select **Ubuntu 24.04 LTS**.
- Choose **t2.micro** or **t3.micro**.



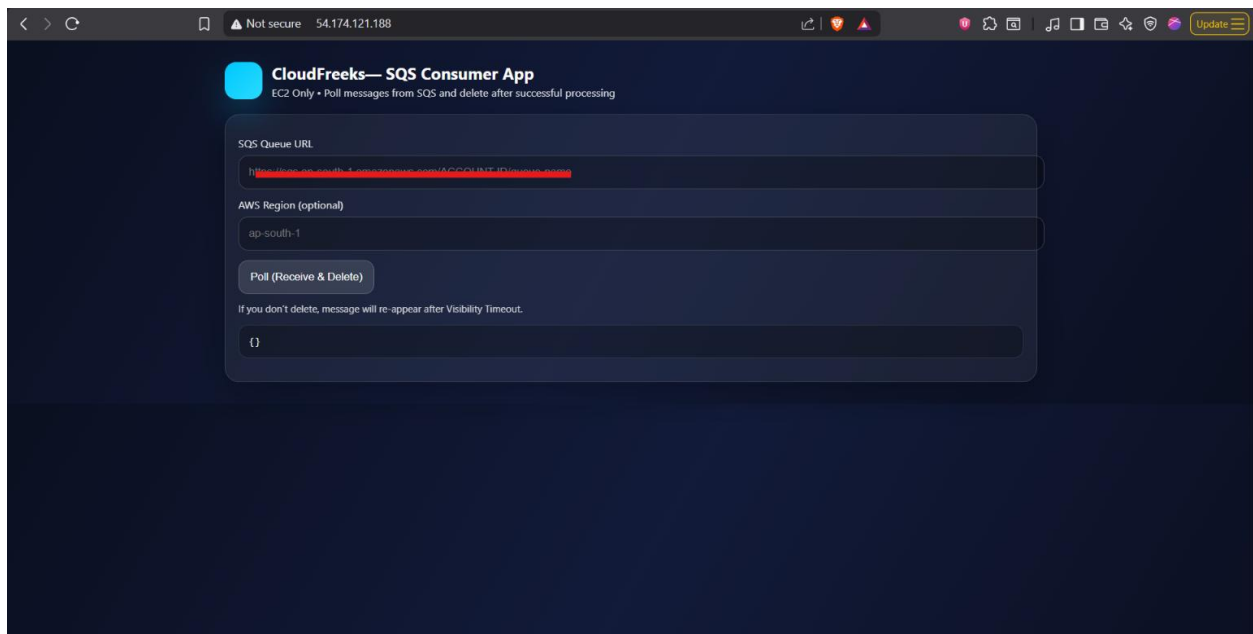
- Select or create a key pair.
- Use the default VPC.
- Allow inbound **SSH 22** and **HTTP 80**.



- In advanced details, select the IAM instance profile **CloudFolks-SQS-EC2-Role**.

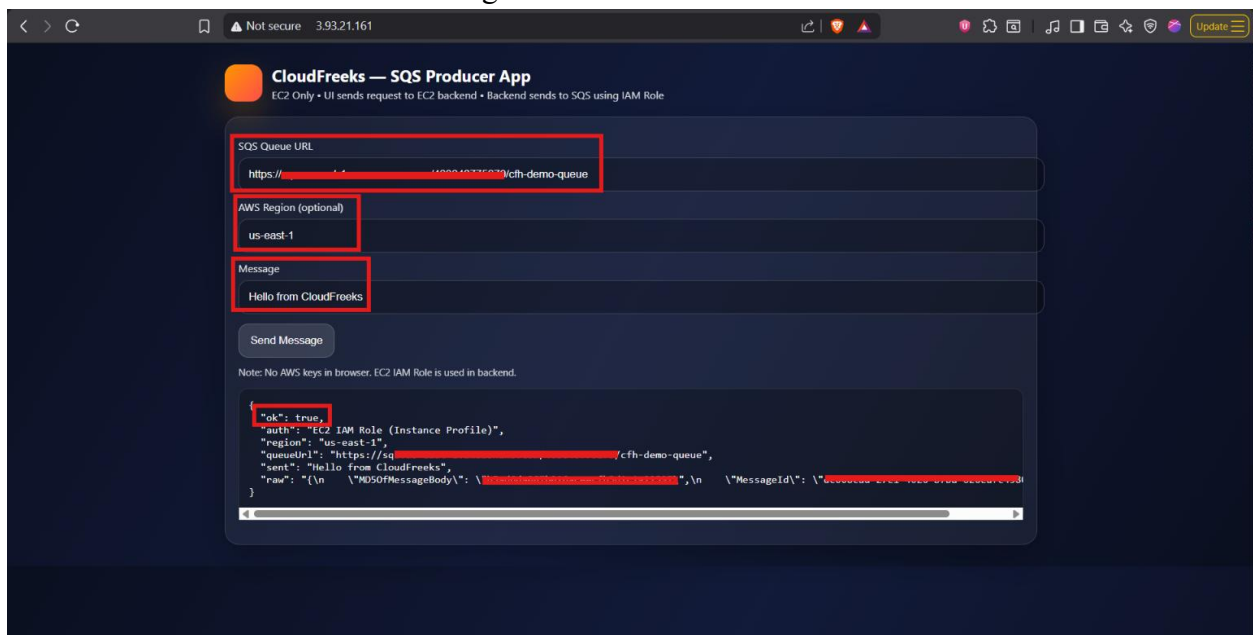


- Add the consumer user data script.
- Launch the instance.
- Wait until the consumer web UI is ready.



Step 5 – Send a Message from the Producer UI

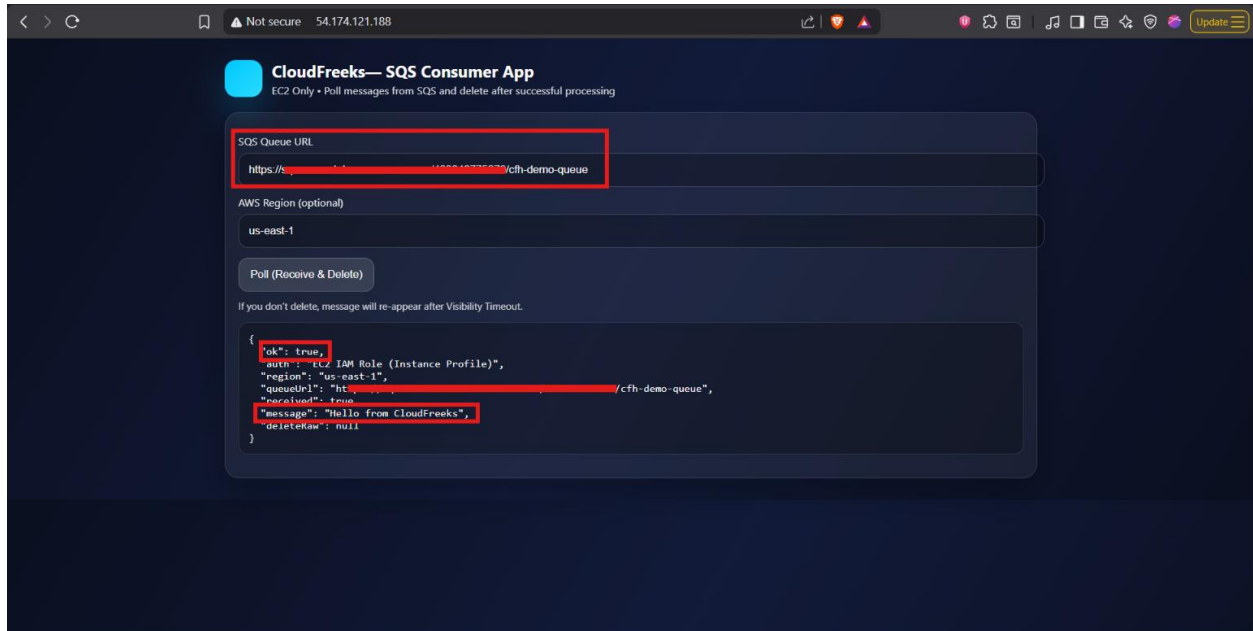
- Copy the public IP address of the producer EC2 instance.
- Open the producer UI in a browser using **HTTP**.
- Paste the **Queue URL** into the producer form.
- Select the correct AWS Region.
- Type a message such as: Hello from CloudFreeks
- Click **Send Message**.
- Confirm that the UI shows message send success.



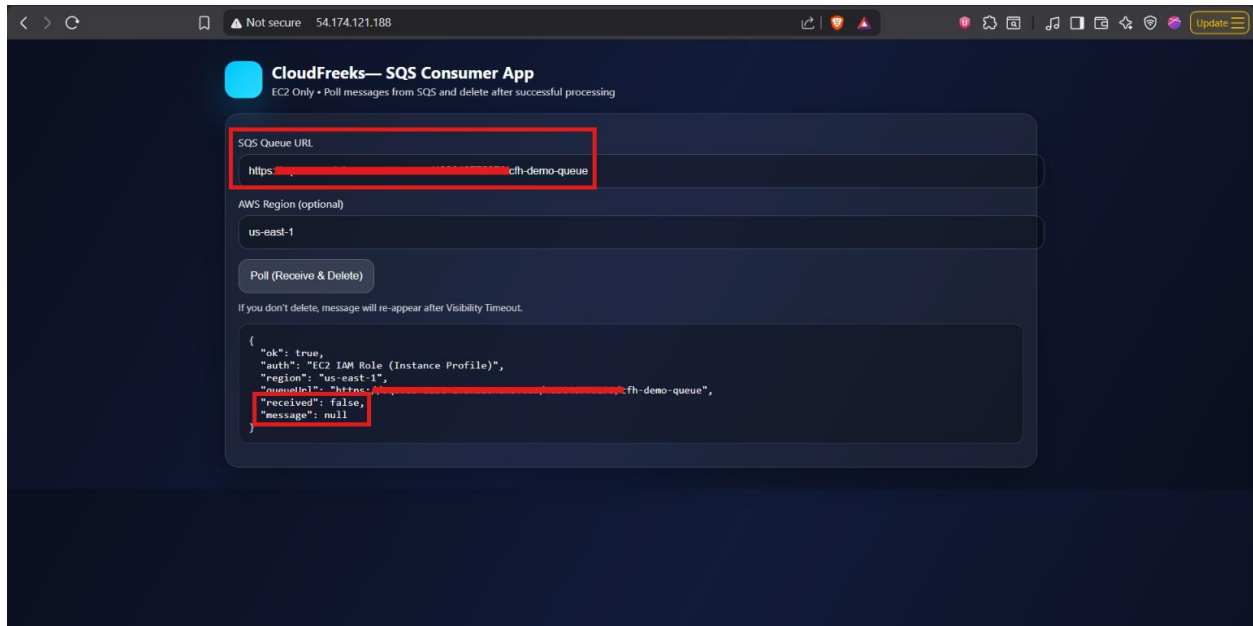
- Check that the message count in SQS increases.

Step 6 – Poll the Message from the Consumer UI

- Copy the public IP address of the consumer EC2 instance.
- Open the consumer UI in a browser using **HTTP**.
- Paste the same **Queue URL**.
- Click **Poll** or **Receive & Delete**.
- Confirm that the message appears in the output.



- Verify that the message is deleted from SQS after polling.



Step 7 – Validate the End-to-End Workflow

- Check that the producer sends messages successfully.

- Check that the consumer receives the message.
- Refresh the queue and confirm that the message is gone after polling.
- Poll again and confirm that there are no messages left.
- Verify that the workflow is pull-based and that SQS never pushes messages itself.

Verification

- An **Amazon SQS Standard Queue** was created successfully.
- A shared **IAM role** was created for EC2 access to SQS.
- The producer EC2 instance sent a message into the queue.
- The consumer EC2 instance pulled and deleted the message from the queue.
- The message count increased and then returned to zero after polling.
- The lab confirmed the complete producer → queue → consumer workflow end to end.